Lab 4: VibeCoding - Chat History

Lunar Lander Repair Lab (Pygame) | Conversation with Claude

Repo: github.com/nehaks28/32_lunarLander

USER

[Uploaded: Lab_4_VibeCoding_Student_handout.pdf, game.py, README.md]
https://github.com/nehaks28/32_lunarLander forked link guide me step by step wt are problems in the existing code wt to change

CLAUDE

Walkthrough based on the uploaded files (the GitHub fork itself was not opened).

Part 1: What is wrong in the code

The bug (Task 1). In `touchdown()` the landing condition checks vertical speed (MAX_SPEED_Y) and angle (MAX_ANGLE) but never checks **MAX_SPEED_X**. The HUD turns Vx red when it is too high, yet the landing still counts as perfect.

```
# Fix
if (pad and abs(self.vel.x) <= MAX_SPEED_X
        and abs(self.vel.y) <= MAX_SPEED_Y
        and abs(angle) <= MAX_ANGLE):
```

The crash-reason logic should also be updated so a sideways crash shows a clear message:

```
if pad is None:
    reason = "missed the pad"
elif abs(angle) > MAX_ANGLE:
    reason = "bad angle"
elif abs(self.vel.x) > MAX_SPEED_X:
    reason = "too much sideways speed"
else:
    reason = "too fast"
```

The three empty functions (Tasks 2-4) all contain only `pass`: ship_color(fuel_ratio) should shift the hull from white to red as fuel drops; on_landing(score) should add an effect after a landing; bonus_life_threshold() should return a score value (e.g. 1500) for extra lives.

Part 2: Step-by-step process for the lab

- **Clone the fork:** git clone, cd, pip install pygame, python game.py.
- **Record the 'before' video** (10 s) showing the bug: landing on a pad with high sideways speed (Vx red) is wrongly 'Perfect landing!'.
- **Keep one continuous LLM chat** since the full history must be exported.
- **Do four tasks with four separate commits** (Task 1 to Task 4), testing before each commit.
- **Record the 'after' video** (10 s): sideways landing now crashes, hull turns red as fuel drops, bonus life at threshold.
- **Add deliverables to a Lab-4 folder** (game.py, videos, chat history) and push.
- **Do not open a PR** to the original SETAPESU26 repo.

Notes: the README asks for a chat link while the handout asks for an exported doc/pdf, so provide both. Pad landing needs the ship body (+/-8 px) fully inside the pad, so test on the wider pad.

USER

wt are changes to make it each task and commands to run after each task code correctio

CLAUDE

Setup (once)

```
git clone https://github.com/nehaks28/32_lunarLander.git
cd 32_lunarLander
pip install pygame
python game.py        # record the "before" video here first
```

Task 1: Fix the touchdown bug

```
# Replace the landing condition in touchdown():
if (pad and abs(self.vel.x) <= MAX_SPEED_X
        and abs(self.vel.y) <= MAX_SPEED_Y
        and abs(angle) <= MAX_ANGLE):

# ...and the crash reason block:
if pad is None:
    reason = "missed the pad"
elif abs(angle) > MAX_ANGLE:
    reason = "bad angle"
elif abs(self.vel.x) > MAX_SPEED_X:
    reason = "too much sideways speed"
else:
    reason = "too fast"

python game.py     # land on a pad with Vx red: "Crashed: too much sideways speed!"
git add game.py
git commit -m "Task 1: Fix touchdown to validate MAX_SPEED_X"
git push origin main
```

Task 2: ship_color(fuel_ratio)

```
def ship_color(fuel_ratio):
    """White at full fuel, fading to red at empty."""
    fuel_ratio = max(0.0, min(1.0, fuel_ratio))
    r = 230
    g = int(40 + (230 - 40) * fuel_ratio)
    b = int(40 + (240 - 40) * fuel_ratio)
    return (r, g, b)

python game.py     # hold Up to burn fuel: ship turns white -> red
git add game.py
git commit -m "Task 2: Implement ship_color fuel-based hull colour"
git push origin main
```

Task 3: on_landing(score) - firework effect

Change A: add above on_landing and replace the function:

```
PARTICLES = []

def on_landing(score):
    """Spawn a firework burst; bigger for high scores."""
    cx, cy = random.randint(150, 650), random.randint(120, 250)
    color = random.choice([(255, 220, 80), (120, 255, 160), (255, 120, 200), (120, 200, 255)])
    for _ in range(60 if score >= 300 else 30):
        a = random.uniform(0, math.tau)
        speed = random.uniform(40, 160)
        PARTICLES.append([cx, cy, math.cos(a) * speed, math.sin(a) * speed, 60, color])
```

Change B: at the end of Game.draw(), after the `if self.message:` block:

```
        for p in PARTICLES[:]:
            p[0] += p[2] / 60
            p[1] += p[3] / 60
            p[3] += 2          # slight gravity
            p[4] -= 1          # lifetime
            if p[4] <= 0:
                PARTICLES.remove(p)
            else:
                pygame.draw.circle(screen, p[5], (int(p[0]), int(p[1])), 3)
```

The particle update lives in draw() because update() returns early once the ship has landed, which would freeze the particles.

```
python game.py     # land safely on a pad: fireworks burst
git add game.py
git commit -m "Task 3: Implement on_landing firework effect"
git push origin main
```

Task 4: bonus_life_threshold()

```
def bonus_life_threshold():
    return 1500

# Quick test: temporarily return 300, cross 300 points, Lives goes up by 1. Then set back to 1500.
python game.py
git add game.py
git commit -m "Task 4: Set bonus life threshold to 1500"
git push origin main
```

Final submission

```
mkdir Lab-4
cp game.py Lab-4/
cp /path/to/before.mp4 /path/to/after.mp4 /path/to/chat_history.pdf Lab-4/
git add Lab-4
git commit -m "Add Lab-4 deliverables: videos, code, chat history"
git push origin main
```

If git push is rejected, run git pull --rebase origin main and push again. If your default branch is master, use that name instead.

USER

do i need to win game cause iam unable to play it in before and after both

CLAUDE

No, you do not need to win. The game has endless levels, and the videos only need to show the bug before and the fix and features after. A temporary demo shortcut lets the ship land on its own.

Temporary demo helper (add at the end of new_round(), only while recording)

```
        # TEMP DEMO HACK - remove before committing
        x1, x2, py, _ = self.pads[0]
        t = math.sqrt(2 * (py - FOOT - 70) / 18)
        self.vel = pygame.Vector2(40, 0)            # change to 0 for a clean landing
        self.pos.x = ((x1 + x2) / 2 - 40 * t) % WIDTH  # use (x1 + x2) / 2 when vel.x is 0
```

Also temporarily set MAX_SPEED_Y = 9999.0, because a free fall is too fast to pass the vertical check.

Before video (10 s)

- Add the hack with vel.x = 40 and the MAX_SPEED_Y change on the original buggy code.
- Run python game.py and do not touch anything. The ship lands with Vx in red and the game shows 'Perfect landing!' - that is the bug.
- Record it.

After video (10 s)

- **Task 1:** with vel.x = 40 it now shows 'Crashed: too much sideways speed!'
- **Task 3:** use vel = Vector2(0, 0) and pos.x = (x1 + x2) / 2: landing succeeds with fireworks.
- **Task 2:** hold Up for a few seconds in a normal run to show the hull turning red.
- **Task 4:** temporarily return 300 from bonus_life_threshold(), land once, Lives goes up by 1.

Before you commit: delete the TEMP DEMO HACK lines, set MAX_SPEED_Y back to 40.0 and the bonus threshold back to 1500. Record the videos first and then commit so the hack never enters a commit.

USER

give me the whole code after all changes are made

CLAUDE

Complete final game.py created with all four tasks applied and no demo hack lines (full listing at the end of this document, Appendix A). Summary:

- **Task 1:** touchdown() now checks MAX_SPEED_X; crash message says 'too much sideways speed'.
- **Task 2:** ship_color() fades the hull from white to red as fuel drops.
- **Task 3:** on_landing() spawns a firework burst; particles are drawn at the end of draw().
- **Task 4:** bonus_life_threshold() returns 1500.

USER

to copy

CLAUDE

Provided the full final code as a single copyable block (same as Appendix A). Replace everything in game.py with it and run python game.py. For separate commits per task, use the task-by-task edits instead of pasting the whole file at once.

USER

see can u generate pdf of this vibecoding convo of us done till now

CLAUDE

Generated this PDF of the conversation.

Appendix A: Final game.py

```
import math
import random
import pygame

WIDTH, HEIGHT = 800, 600
STEP = 20
THRUST, ROTATE_SPEED, BURN_RATE, FUEL_MAX = 60.0, 2.4, 22.0, 300.0
MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25
FOOT = 12

# Firework particles: [x, y, vx, vy, lifetime, color]
PARTICLES = []


def ship_color(fuel_ratio):
    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full): white at full fuel, red at empty."""
    fuel_ratio = max(0.0, min(1.0, fuel_ratio))
    r = 230
    g = int(40 + (230 - 40) * fuel_ratio)
    b = int(40 + (240 - 40) * fuel_ratio)
    return (r, g, b)


def on_landing(score):
    """Called after a successful landing with the points just earned; spawns a firework burst."""
    cx, cy = random.randint(150, 650), random.randint(120, 250)
    color = random.choice([(255, 220, 80), (120, 255, 160), (255, 120, 200), (120, 200, 255)])
    for _ in range(60 if score >= 300 else 30):
        a = random.uniform(0, math.tau)
        speed = random.uniform(40, 160)
        PARTICLES.append([cx, cy, math.cos(a) * speed, math.sin(a) * speed, 60, color])


def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
    return 1500


def make_terrain():
    heights, y = [], random.randint(430, 520)
    for _ in range(WIDTH // STEP + 1):
        y = max(380, min(560, y + random.randint(-32, 32)))
        heights.append(y)
    pads = []
    for start, length, mult in ((random.randint(2, 12), 4, 1), (random.randint(20, 35), 3, 3)):
        for i in range(start, start + length):
            heights[i] = heights[start]
        pads.append((start * STEP, (start + length - 1) * STEP, heights[start], mult))
    return heights, pads


def ground_y(heights, x):
    x = max(0, min(WIDTH - 1, x))
    i = int(x // STEP)
    t = (x - i * STEP) / STEP
    return heights[i] * (1 - t) + heights[i + 1] * t


def wrap_angle(angle):
    return (angle + math.pi) % math.tau - math.pi


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.level, self.score, self.lives = 1, 0, 3
        self.bonus_awarded = 0
        self.new_round()

    def new_round(self):
        self.heights, self.pads = make_terrain()
        self.pos = pygame.Vector2(random.randint(100, 700), 70)
        self.vel = pygame.Vector2(random.uniform(-20, 20), 0)
        self.angle, self.fuel, self.thrusting = 0.0, FUEL_MAX, False
```

```
    self.state, self.message = "fly", ""

def pad_under(self):
    for x1, x2, y, mult in self.pads:
        if x1 <= self.pos.x - 8 and self.pos.x + 8 <= x2:
            return (x1, x2, y, mult)
    return None

def touchdown(self):
    pad = self.pad_under()
    angle = wrap_angle(self.angle)
    # Task 1 fix: also validate horizontal speed against MAX_SPEED_X
    if (pad and abs(self.vel.x) <= MAX_SPEED_X
            and abs(self.vel.y) <= MAX_SPEED_Y
            and abs(angle) <= MAX_ANGLE):
        earned = int((100 + self.fuel) * pad[3])
        self.score += earned
        self.state, self.message = "landed", f"Perfect landing! +{earned}  (Space = next level)"
        on_landing(earned)
        return
    self.lives -= 1
    self.state = "crashed"
    if pad is None:
        reason = "missed the pad"
    elif abs(angle) > MAX_ANGLE:
        reason = "bad angle"
    elif abs(self.vel.x) > MAX_SPEED_X:
        reason = "too much sideways speed"
    else:
        reason = "too fast"
    self.message = f"Crashed: {reason}!  " + ("Space = retry" if self.lives > 0 else "Game over - R = restart")

def update(self, dt, keys):
    if self.state != "fly":
        return
    threshold = bonus_life_threshold()
    if threshold and self.score // threshold > self.bonus_awarded:
        self.bonus_awarded = self.score // threshold
        self.lives += 1
    self.angle += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * ROTATE_SPEED * dt
    gravity = pygame.Vector2(0, 16 + 2 * self.level)
    self.thrusting = bool(keys[pygame.K_UP]) and self.fuel > 0
    acceleration = gravity
    if self.thrusting:
        acceleration = gravity + pygame.Vector2(math.sin(self.angle), -math.cos(self.angle)) * THRUST
        self.fuel = max(0.0, self.fuel - BURN_RATE * dt)
    self.vel += acceleration * dt
    self.pos += self.vel * dt
    self.pos.x %= WIDTH
    self.pos.y = max(-200, self.pos.y)
    if self.pos.y + FOOT >= ground_y(self.heights, self.pos.x):
        self.touchdown()

def ship_points(self):
    cos, sin = math.cos(self.angle), math.sin(self.angle)
    local = [(0, -16), (10, 10), (-10, 10)]
    return [(self.pos.x + x * cos - y * sin, self.pos.y + x * sin + y * cos) for x, y in local]

def draw(self, screen):
    screen.fill((8, 8, 20))
    points = [(i * STEP, h) for i, h in enumerate(self.heights)]
    pygame.draw.polygon(screen, (70, 70, 85), points + [(WIDTH, HEIGHT), (0, HEIGHT)])
    pygame.draw.lines(screen, (190, 190, 200), False, points, 2)
    for x1, x2, y, mult in self.pads:
        pygame.draw.line(screen, (90, 230, 120), (x1, y), (x2, y), 5)
        label = self.font.render(f"x{mult}", True, (90, 230, 120))
        screen.blit(label, label.get_rect(midtop=((x1 + x2) / 2, y + 8)))
    if self.state != "crashed":
        if self.thrusting:
            cos, sin = math.cos(self.angle), math.sin(self.angle)
            flame = [(self.pos.x - 5 * cos - 10 * sin, self.pos.y - 5 * sin + 10 * cos),
                     (self.pos.x + 5 * cos - 10 * sin, self.pos.y + 5 * sin + 10 * cos),
                     (self.pos.x - 22 * sin, self.pos.y + 22 * cos)]
            pygame.draw.polygon(screen, (255, 170, 40), flame)
        color = ship_color(max(0.0, self.fuel) / FUEL_MAX) or (230, 230, 240)
        pygame.draw.polygon(screen, color, self.ship_points())
    else:
        pygame.draw.circle(screen, (255, 120, 40), self.pos, 24, 3)
    ok_x = abs(self.vel.x) <= MAX_SPEED_X
    ok_y = abs(self.vel.y) <= MAX_SPEED_Y
```

```
        ok_a = abs(wrap_angle(self.angle)) <= MAX_ANGLE
        good, bad = (120, 240, 140), (250, 110, 100)
        lines = [
            (f"Fuel {self.fuel:5.0f}", (240, 240, 240)),
            (f"Vx {self.vel.x:6.1f}", good if ok_x else bad),
            (f"Vy {self.vel.y:6.1f}", good if ok_y else bad),
            (f"Angle {math.degrees(wrap_angle(self.angle)):5.0f}", good if ok_a else bad),
            (f"Score {self.score}  Lives {self.lives}  Level {self.level}", (240, 240, 240)),
        ]
        for i, (text, color) in enumerate(lines):
            screen.blit(self.font.render(text, True, color), (10, 8 + i * 22))
        if self.message:
            label = self.font.render(self.message, True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3)))
        # Task 3: draw and advance firework particles (here so they keep moving after landing)
        for p in PARTICLES[:]:
            p[0] += p[2] / 60
            p[1] += p[3] / 60
            p[3] += 2          # slight gravity
            p[4] -= 1          # lifetime
            if p[4] <= 0:
                PARTICLES.remove(p)
            else:
                pygame.draw.circle(screen, p[5], (int(p[0]), int(p[1])), 3)


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Lunar Lander")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    game.reset()
                elif event.key == pygame.K_SPACE and game.state == "landed":
                    game.level += 1
                    game.new_round()
                elif event.key == pygame.K_SPACE and game.state == "crashed" and game.lives > 0:
                    game.new_round()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
    pygame.quit()


if __name__ == "__main__":
    main()
```